

Introduction to Computer Architecture

Chapter 2

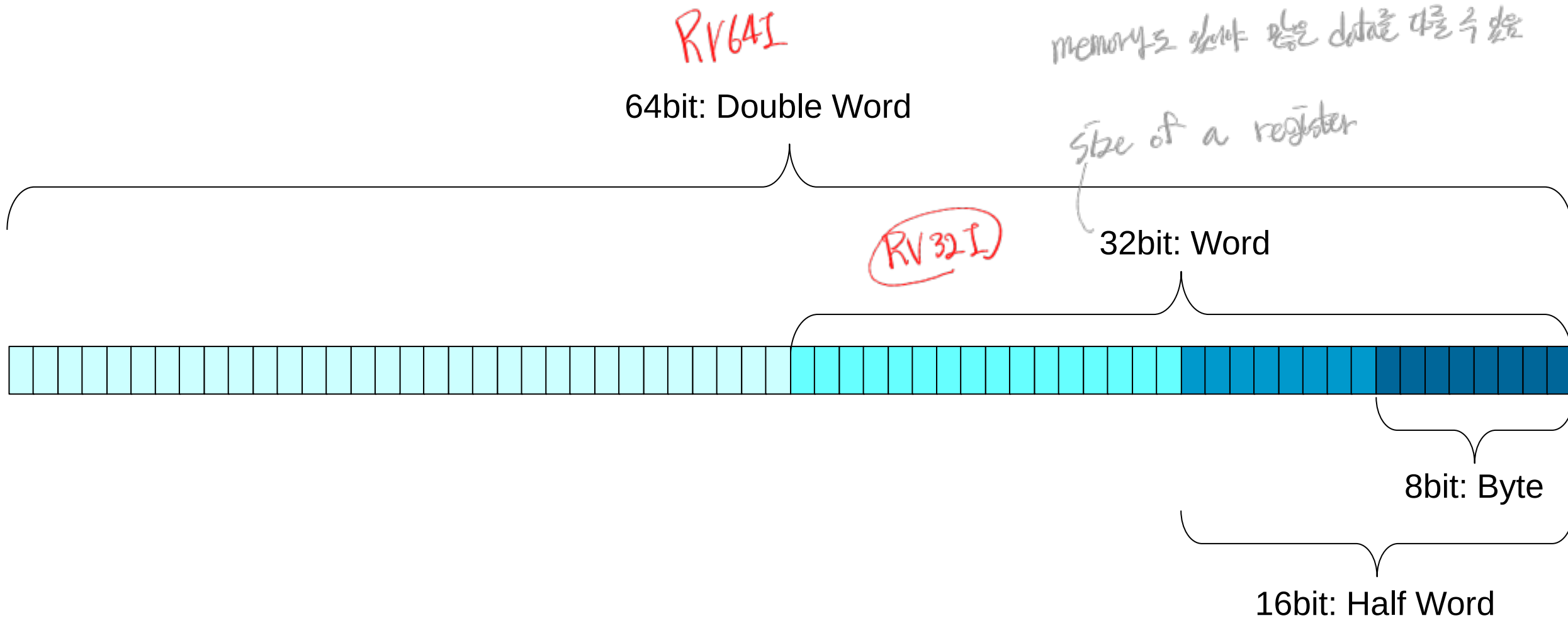
Instructions: Language of the Computer - 2

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

MEMORY INSTRUCTIONS

Data Size



Memory Addressing

1byte (8-bit)

e.g., $00110110_2 = 0x36$
1 byte

Address: 0x1000 0x1001 0x1002

Memory:

32bit: Word

destination reg source reg

- Memory “address” is also a 32-bit (non-negative) integer.

❖ 0x0000_0000 – 0xFFFF_FFFF

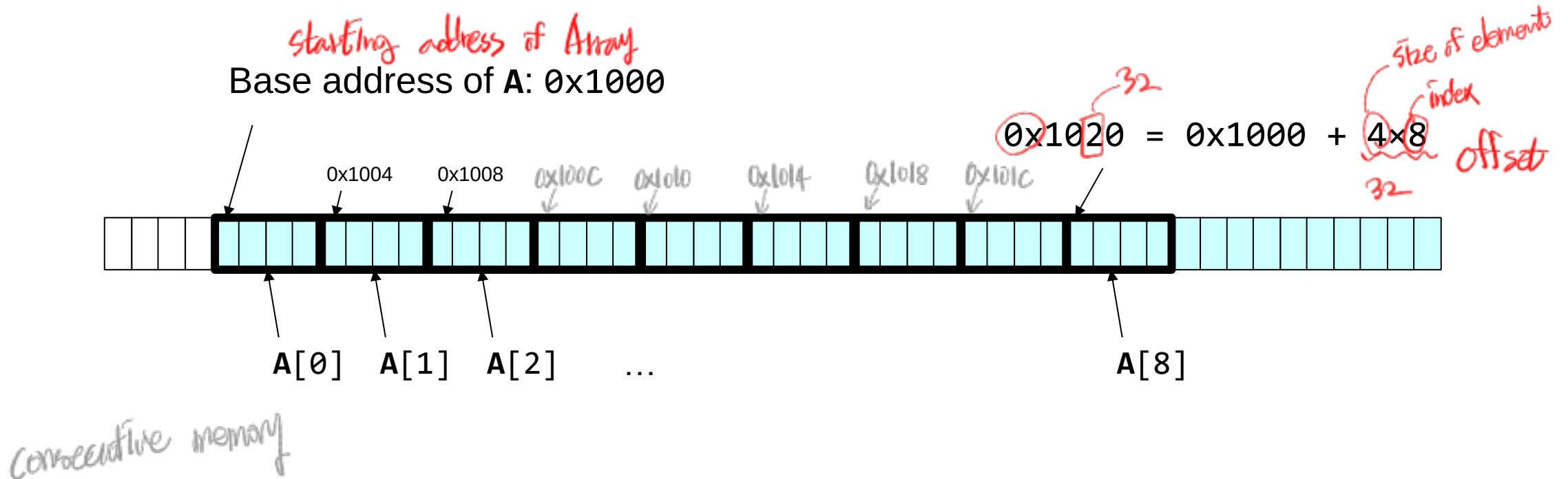
- Memory → register: **Load** Need address & destination register *rd*
- Register → memory: **Store** Need address & source register *rs*

Memory Operand Example 1

■ C code:

```
g = h + A[8];
```

- ❖ g in x1, h in x2, base address (starting address) of A in x3
- ❖ A is an 'int' type array (i.e., each element is a 4-byte value)



Memory Operand Example 1

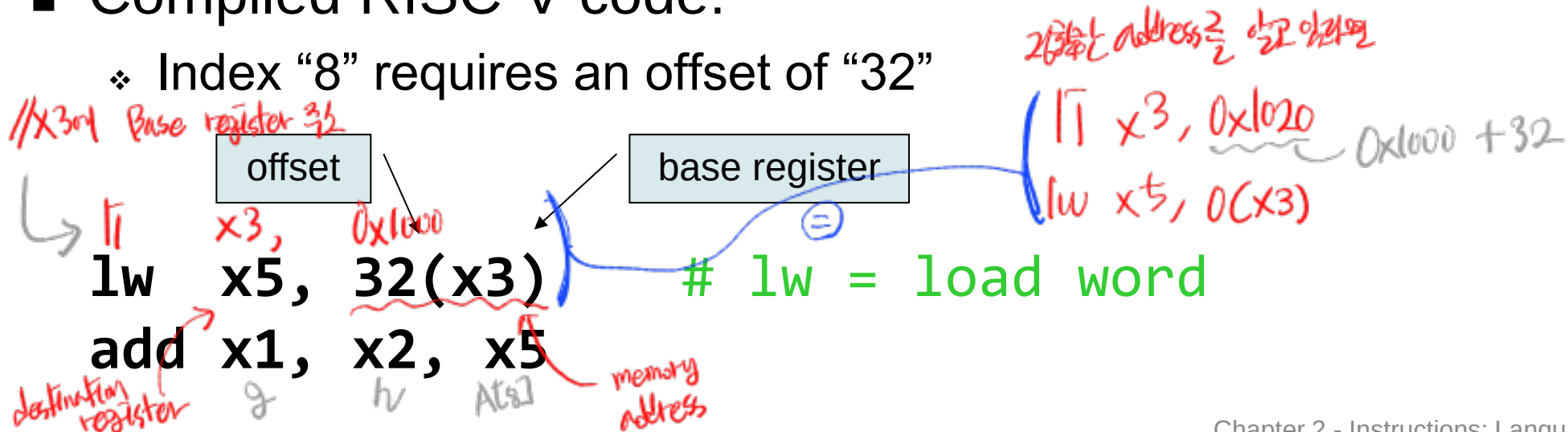
■ C code:

```
g = h + A[8];
```

- ❖ g in x1, h in x2, base address (starting address) of A in x3
- ❖ A is an 'int' type array (i.e., each element is a 4-byte value)

■ Compiled RISC-V code:

- ❖ Index "8" requires an offset of "32"



Memory Operand Example 2

■ C code:

`A[12] = h + A[8];`

❖ h in x2, base address of A in x3

■ Compiled RISC-V code:

```
lw  x6, 32(x3)    # load word
add x7, x2, x6
sw  x7, 48(x3)    # store word
```

Base address가 같을 때
offset만 바뀌면 될
(이 2개 instruction)

```
( li x3, 0x1020
  lw x6, 0(x3)
  add x7, x2, x6
  li x3, 0x1030
  sw x7, 0(x3) )
```

Memory Operand Example 3

■ C code:

```
for( i = 0; i < 10; i++ ) {  
    sum = sum + A[i];  
}
```

❖ sum in x1, i in x2, base address of A in x3

■ Compiled RISC-V code (only the loop body):

...

lw x5, 0(x3) → 0x1000+0 i=0
add x1, x1, x5 → 0x1004+0 i=1
addi x3, x3, 4 → 0x1008+0 i=2

...

→ sum = sum + A[i]

RISC-V Instructions – Memory

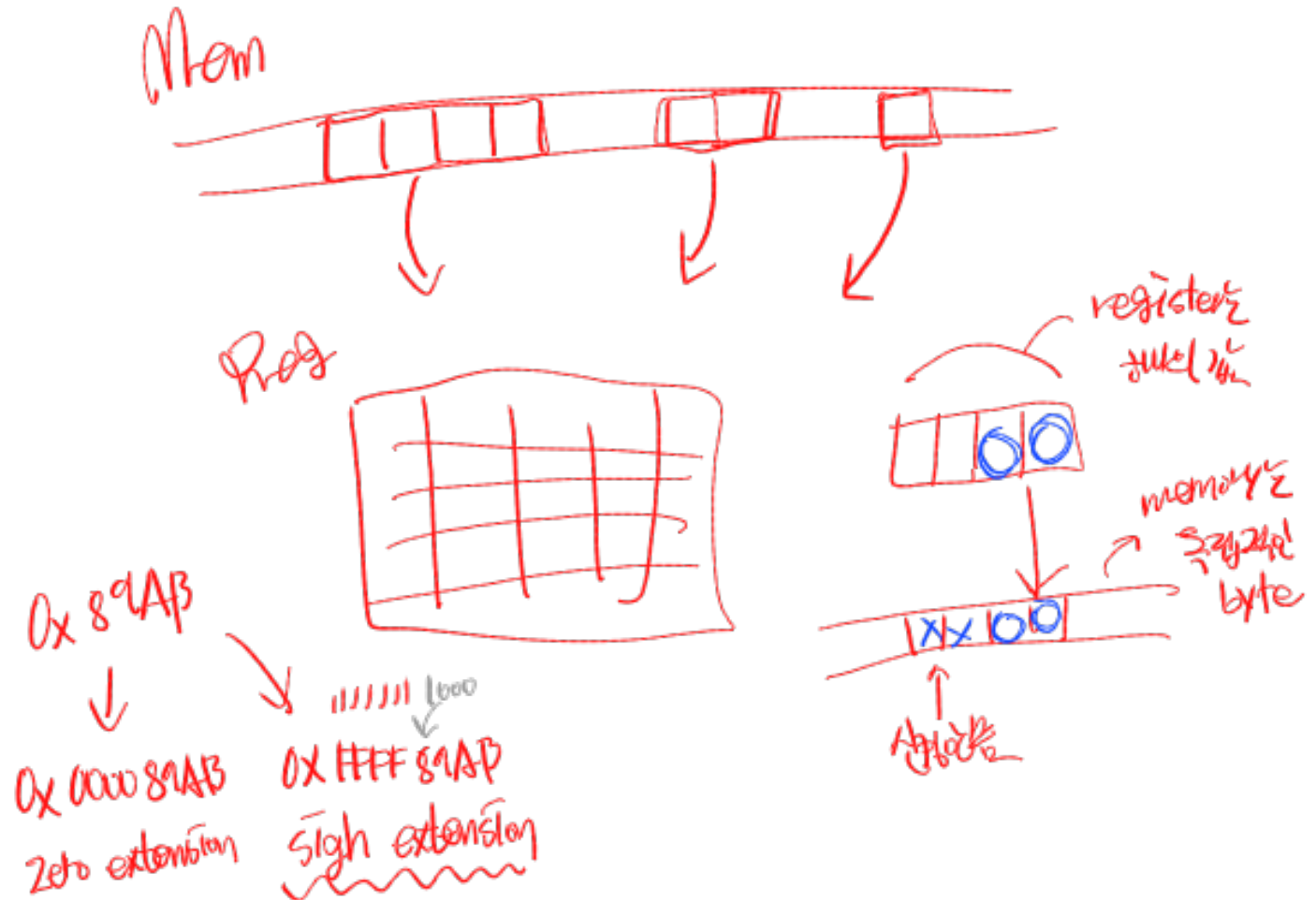
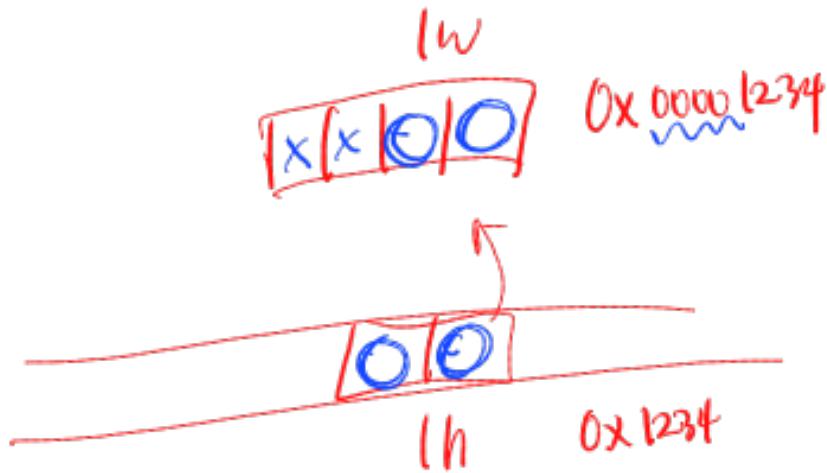
■ lw, lh, lb

- ❖ Load word / halfword / byte

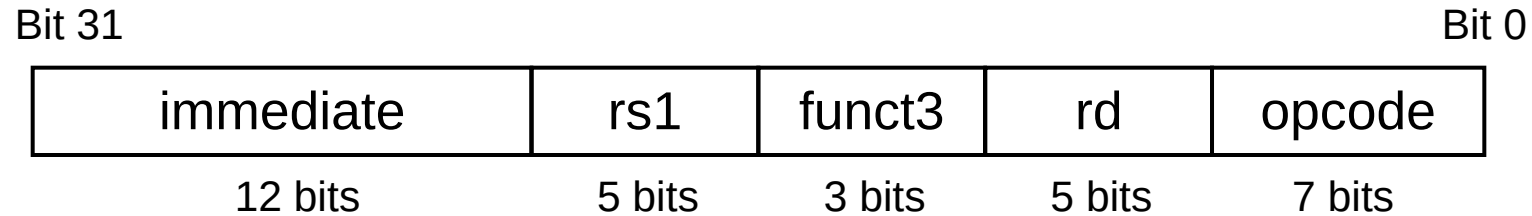
4 byte 2 byte

■ sw, sh, sb

- ❖ Store word / halfword / byte



RISC-V I-format Instructions



opcode *rd* *Imm* *rs1*
lw x9, -123(x20)

immediate	x20	funct3	x9	Load
-----------	-----	--------	----	------

-123	20	2	9	3
------	----	---	---	---

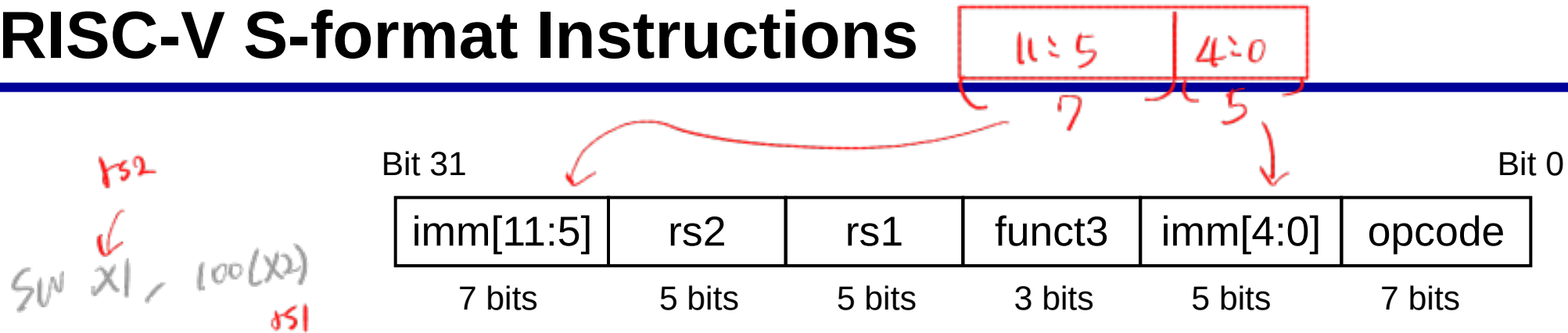
1111 1000 0101	10100	010	01001	0000011
----------------	-------	-----	-------	---------

111110000101 10100 010 01001 0000011₂

1111 1000 0101 1010 0010 0100 1000 0011₂

= 0xF85A2483

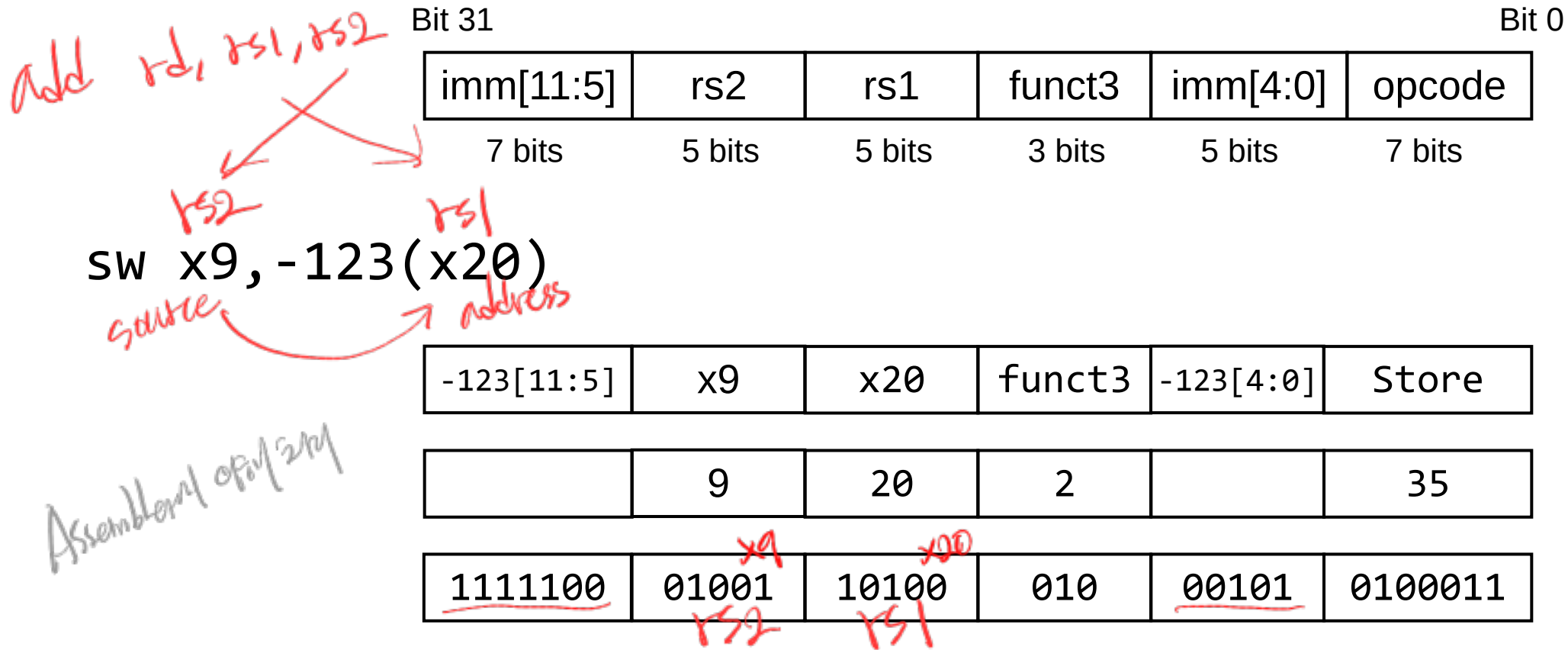
RISC-V S-format Instructions



■ Store instructions use a different binary format:

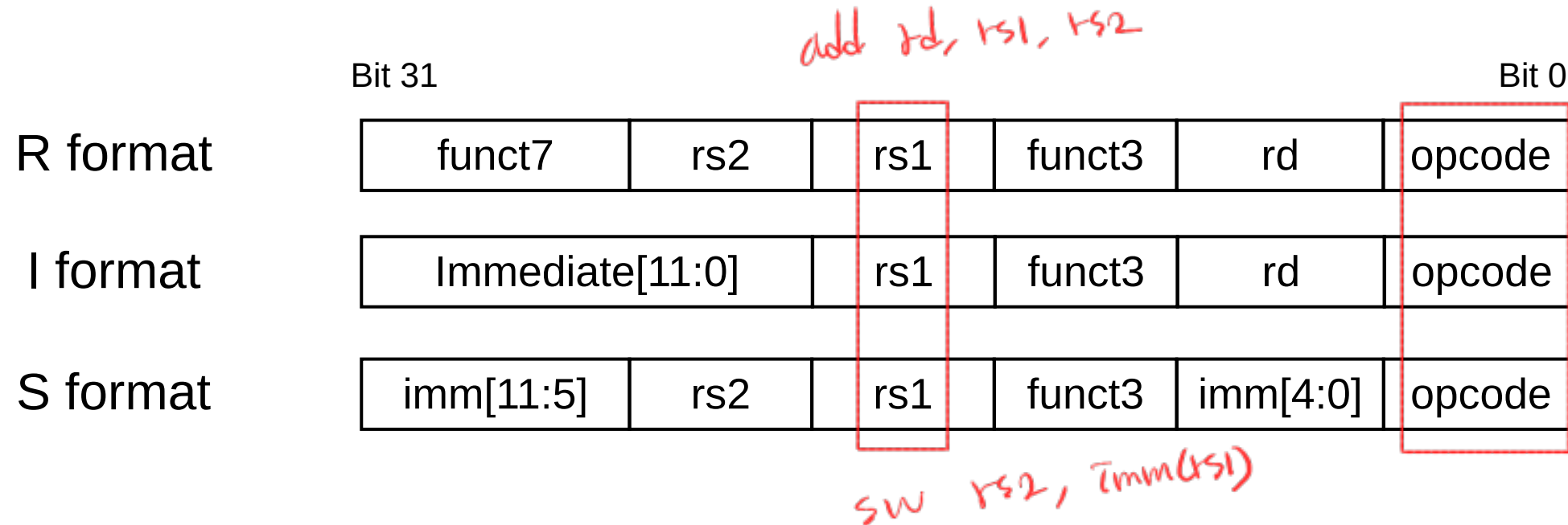
- ❖ rs1: base address register number
- ❖ rs2: source operand register number
- ❖ immediate: offset added to base address
 - Designed so that rs1 and rs2 fields are always in the same place

RISC-V S-format Instructions



1111100 01001 10100 010 00101 0100011₂
 1111 1000 1001 1010 0010 0010 1010 0011₂
 = 0xF89A22A3

RISC-V Machine Code Encoding Comparison



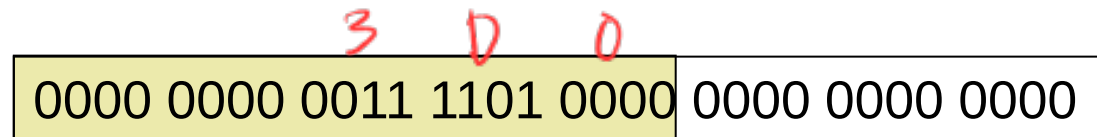
32-bit Constants

- Most constants are small
 - ❖ 12-bit immediate is sufficient for common cases
- For occasional 32-bit constant, use **lui** (Load upper immediate)

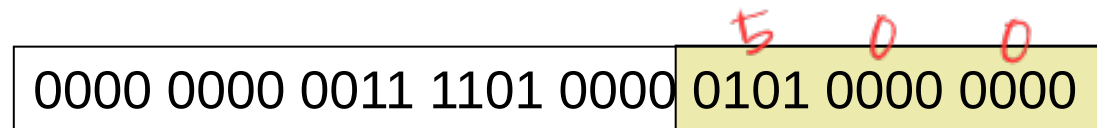
`lui rd, constant`

- ❖ Copies a 20-bit constant to [31:12] of rd
- ❖ Sets the lower 12 bits of rd to 0
- ❖ e.g., `lui x10, 0x123` → x10 becomes 0x00123000

`lui x19, 976 //0x003D0`



`addi x19, x19, 1280 //0x500`



x19: 0x3D0500

LUI Example

RARS RISC-V Simulator

The screenshot displays the RARS RISC-V Simulator interface. The main window shows the assembly file `riscv1.asm` with the following instruction:

```
1 li x10, 0x12345678
```

Handwritten red annotations explain the instruction:

- A red arrow points to the `0x12345678` value, with the text "32 Constant value" and "immediate value".
- Below it, the text "0x12345678 is 32 bits, like pseudo instruction" is written.
- At the bottom, "2nd Inst. 32 bits" is written.

The right-hand panel shows the "Control and Status" window, which includes a "Registers" table. The table lists 32 registers (zero, ra, sp, gp, tp, t0, t1, t2, s0, s1, a0, a1, a2, a3, a4, a5, a6, a7, s2, s3, s4, s5, s6, s7, s8, s9) and their current values, all of which are `0x00000000`.

The bottom status bar indicates "Line: 1 Column: 3" and "Show Line Numbers" is checked. The "Messages" and "Run I/O" buttons are visible at the bottom.

LUI Example

G:\My Drive\Lectures\CompArch\riscv1.asm - RARS 1.5

File Edit Run Settings Tools Help

0101

Edit Execute

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x12345537	lui x10,0x00012345	l: li x10, 0x12345678
	0x00400004	0x67850513	addi x10,x10,0x00000678	

Handwritten red text:
lui x10, 0x12345678
addi의 한계 때문 (최대 12비트)

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+1...	Value (+...	Value (+1...	Value (+...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...
0x100...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...	0x000...

0x10010000 (.data) Hexadecimal Addresses Hexadecimal Values

Messages Run I/O

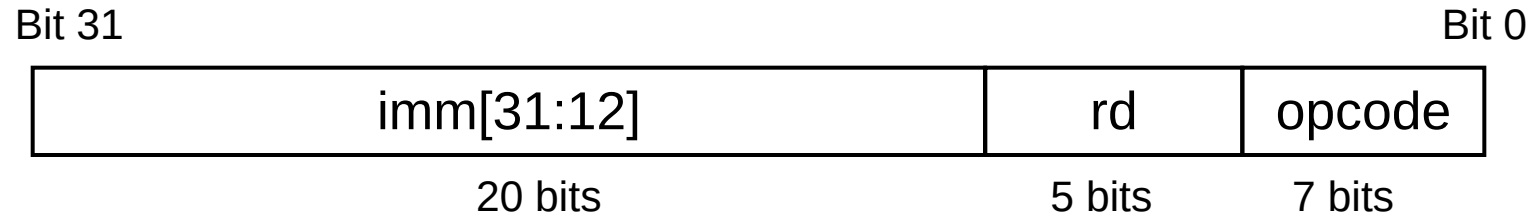
Control and Status

Registers Floating Point

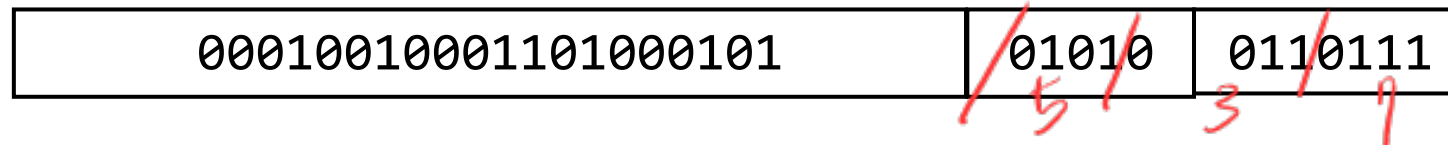
Name	Num...	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7ffffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000

Handwritten red text:
0x12345000 + 0x678

LUI Binary Encoding – U format



lui x10, 0x12345



00010010001101000101 01010 0110111₂
 0001 0010 0011 0100 0101 0101 0011 0111₂
 = 0x12345537

Sign Extension

- RISC-V instructions sign-extend the immediate value

`li t0, 11`

`addi t1, t0, -10`

- **lh** and **lb** also sign-extends the loaded halfword or byte

- **lhu** and **lbu** zero-extends the loaded halfword or byte
unsigned

- There are no **shu** or **sbu** instructions!

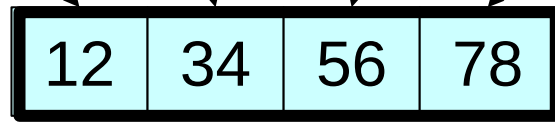
Endianness

What happens if store this 4-byte value to address 0x1000 ?

$$12\ 34\ 56\ 78_{16} = 1 \times 16^7 + 2 \times 16^6 + 3 \times 16^5 + \dots + 7 \times 16^1 + 8 \times 16^0$$

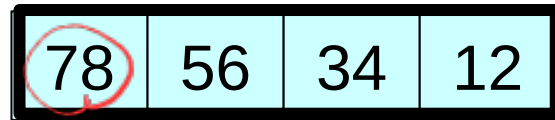
higher digit lower digit
2바이트씩 늘려가: 0x12 34 56 78

lower 0x1000 0x1001 0x1002 0x1003 higher



Big Endian : MIPS, SPARC

↓ reverse



Little Endian : x86, **RISC-V**

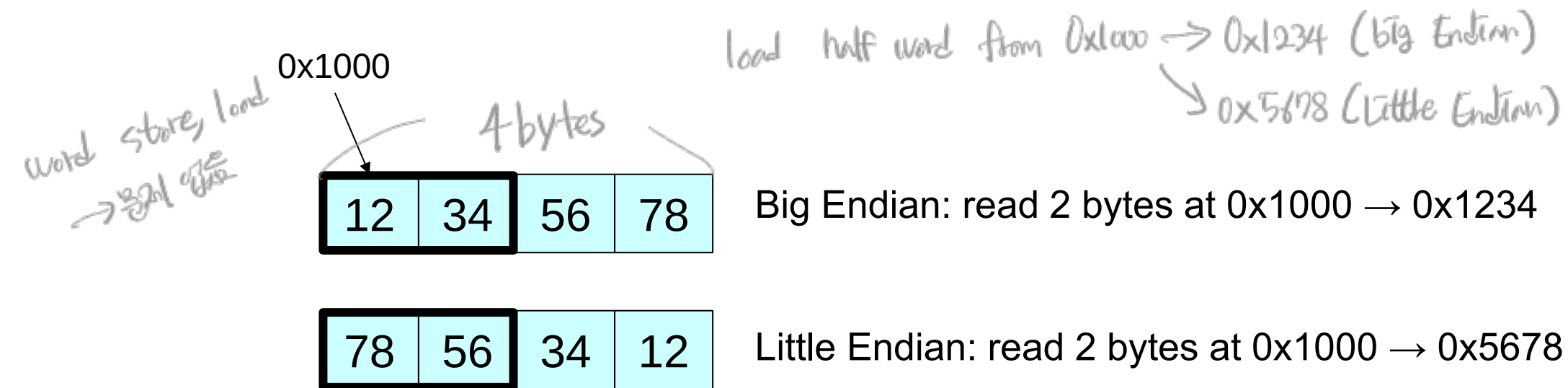
← lower lower digit
higher higher digit →

design choice

less intuitive
높은 주소를 higher digit에
저는 lower digit에 저장

Endianness

- Endianness does not matter when storing/loading 4 bytes at a time at 4-byte aligned addresses.
- It matters when you read in smaller sizes.

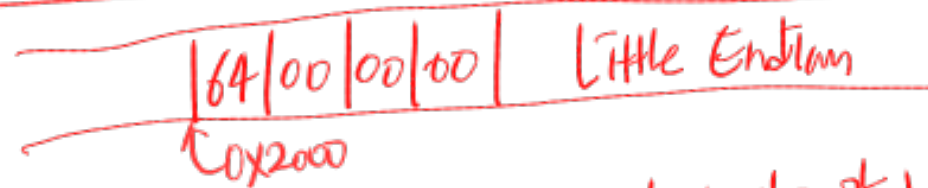


SW "100" 0x64 → 0x 0000 00 64

load byte
↓



if big endian, 1b from 0x2000 → 0x00



RISC →

if little endian,

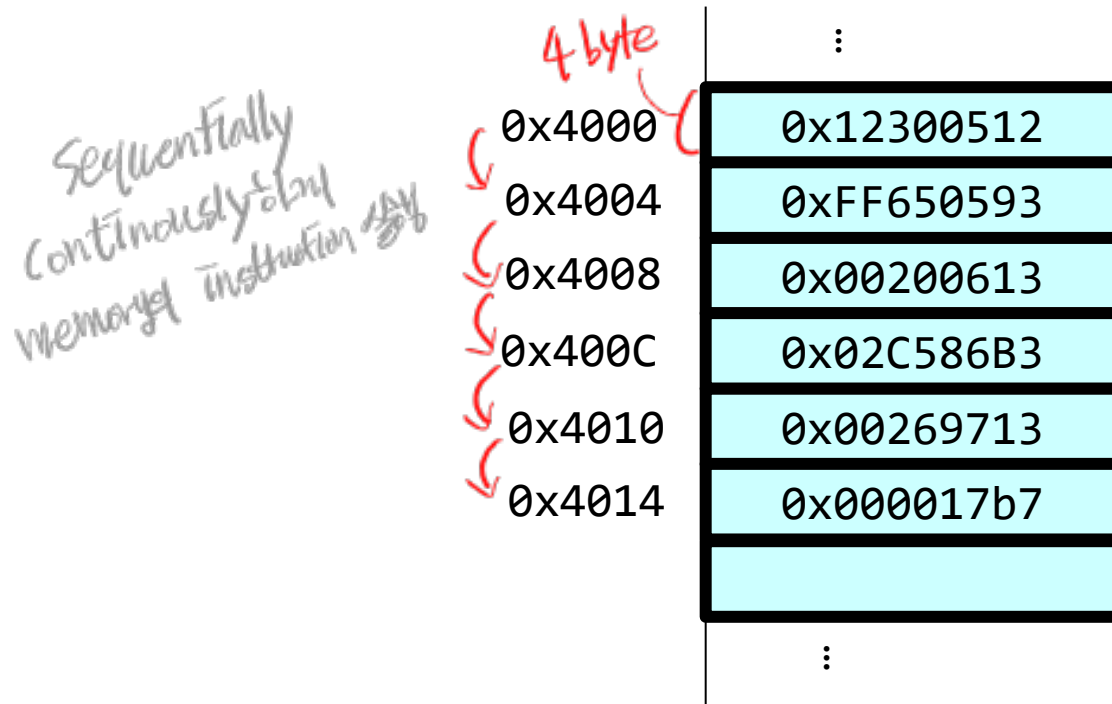
1b

from 0x2000 → 0x64 (store 1 byte at a time)

CONTROL FLOW INSTRUCTIONS

Program Counter

- Program Counter (PC) *register $\neq$ X0~X31*
 - ❖ Indicates the memory address of the current instruction



0 4 8 C 0 4 8 C

```

addi x10, x0, 0x123
addi x11, x10, -10
addi x12, x0, 2
mul  x13, x11, x12
slli x14, x13, 2
lui  x15, 1
    
```

0b00 0
0100 4
1000 8
1100 C
10000 0
10100 4

beq
add
add
li sub

x1, x2, 4
+3x4
+12 00001100₂
+3 000011₂
14bH

이런 식으로 계속 0이냐 1이냐를 136H

Conditional Operations

- Branch to a labeled instruction if a condition is true
 - ❖ Otherwise, continue to the next instruction..

branch equal

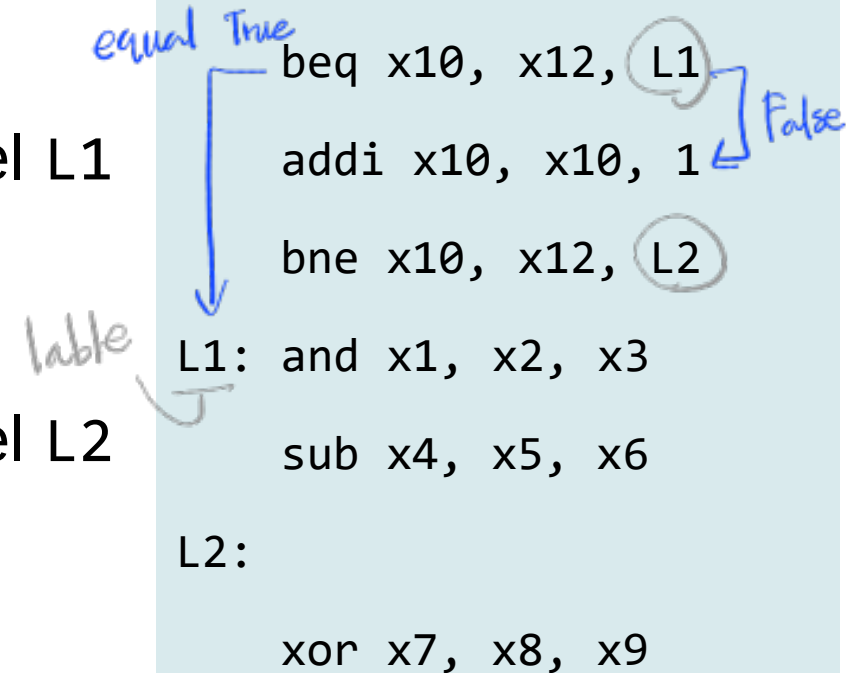
- **beq rs1, rs2, L1**

- ❖ if(*rs1 == *rs2), jump to the instruction at label L1

branch not-equal

- **bne rs1, rs2, L2**

- ❖ if(*rs1 != *rs2), jump to the instruction at label L2



More Conditional Operations

branch less than

■ **blt** rs1, rs2, L3

❖ if(*rs1 < *rs2), jump to the instruction at label L3

branch greater equal

■ **bge** rs1, rs2, L4

❖ if(*rs1 >= *rs2), jump to the instruction at label L4

rs1 값을 < rs2보다 작을
rs1 값을 ≥ rs2보다 크거나 같을

Signed vs. Unsigned

- Signed comparison: **blt**, **bge** *Two's complement*

- Unsigned comparison: **bltu**, **bgeu**
u suffix

*Int32 Unsigned Int32
Adding 2's complement*

- Example

❖ x22 = 1111 1111 1111 1111 1111 1111 1111 1111 *0xFFFF FFFF → -1*
❖ x23 = 0000 0000 0000 0000 0000 0000 0000 0001 *→ 1* *→ 2³²-1*

❖ x22 < x23 *// signed comparison*

➢ -1 < +1

0001 → 1

❖ x22 > x23 *// unsigned comparison*

➢ +4,294,967,295 > +1

Compiling If Statements

■ C code:

```
if (i==j) {  
    f = g + h;  
}  
else {  
    f = g - h;  
}  
f = f << 1;
```

f, g, h, i, j in
x19, x20, x21, x22, x23

■ RISC-V code:

```
bne x22, x23, Else  
add x19, x20, x21 //f=g+h;  
beq x0, x0, Exit //unconditional  
Else:  
sub x19, x20, x21 //f=g-h;  
Exit: slli x19, x19, 1 //f=f<<1;
```

Handwritten notes and annotations:

- equal: branch을 무시* (equal: ignore branch)
- not equal* (not equal)
- 이제 없으면 아래로* (if not, go down)
- Else:까지 실행하게 됨* (will be executed until Else:)
- shift left* (shift left)

Flowchart labels: True, False

Compiling If Statements

■ C code:

```
if (i < 10) {  
    f = g + h;  
}  
else {  
    f = g - h;  
}  
f = f << 1;
```

f, g, h, i, j in
x19, x20, x21, x22, x23

~~bge~~: 상수보다 크거나 같으면
실행하지 않음,
instruction 21B-부터

■ RISC-V code:

```
addi x10, x0, 10
```

```
bge x22, x10, Else
```

```
add x19, x20, x21 //f=g+h;
```

```
j Exit
```

Else:

```
sub x19, x20, x21 //f=g-h;
```

```
Exit: slli x19, x19, 1 //f=f<<1;
```

label과 instruction을 같은 줄에 쓰는 건 상관X

True

False

Compiling Loop Statements

■ C code:

```
k = 0;
while (i >= 0) {
    i += save[k];
    k++;
}
```

i in x22, ~~k~~ in x24

save is an **int** array

base address of save in x25

■ RISC-V code:

```
add x24, x0, x0
```

```
loop: blt x22, x0, loop_end
```

```
slli x11, x24, 2
```

$k \ll 2 \ (k \times 4)$

```
add x10, x11, x25
```

base + 4k

```
lw x10, 0(x10)
```

load

```
add x22, x22, x10
```

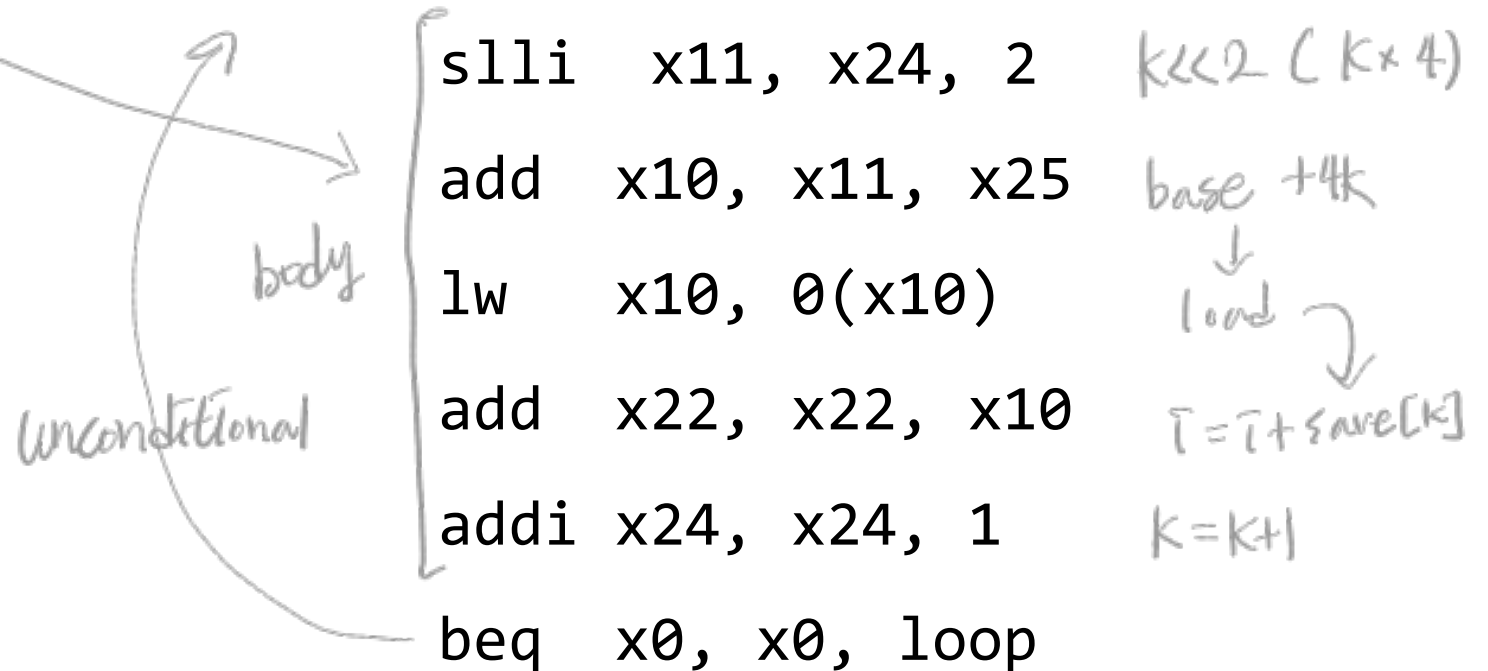
$i = i + \text{save}[k]$

```
addi x24, x24, 1
```

$k = k + 1$

```
beq x0, x0, loop
```

```
loop_end: ...
```



Compiling Loop Statements

optimization

위에서는 1을 2로 4배한 것 대입

■ C code:

```
k = 0;
while (i >= 0) {
    i += save[k];
    k++;
}
```

i in x22, k*4 in x24

save is an **int** array

base address of save in x25

■ RISC-V code:

예에서는 바이트 4를 리딩

0, 4, 8, 12, 16

```
add x24, x0, x0
```

```
loop: blt x22, x0, loop_end
```

```
add x10, x24, x25
```

```
lw x10, 0(x10)
```

```
add x22, x22, x10
```

```
addi x24, x24, 4
```

```
beq x0, x0, loop
```

```
loop_end: ...
```

Back
ward
-5*4

Forward 6*4

← PC

base + k

↓
load

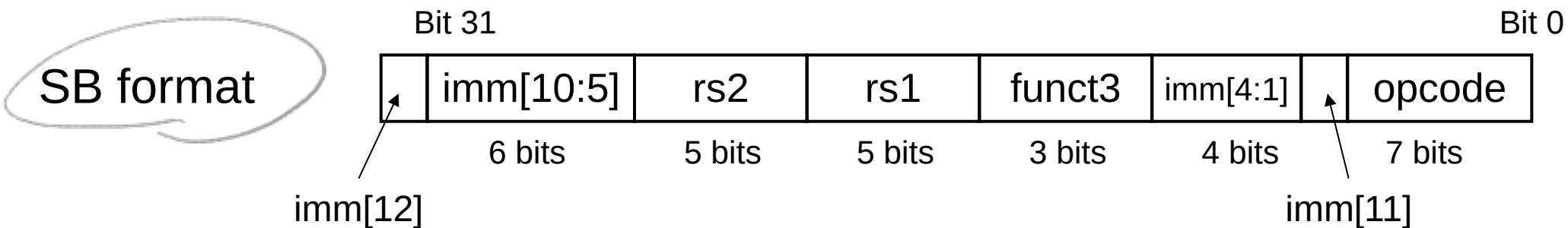
↓
i = i + save[k]

k = k + 4

← PC + 6*4

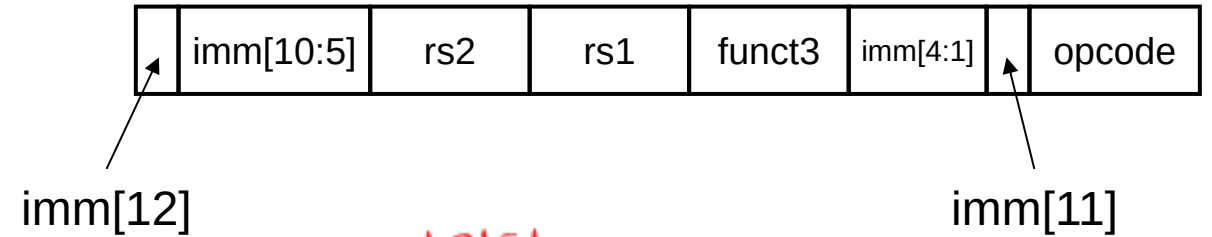
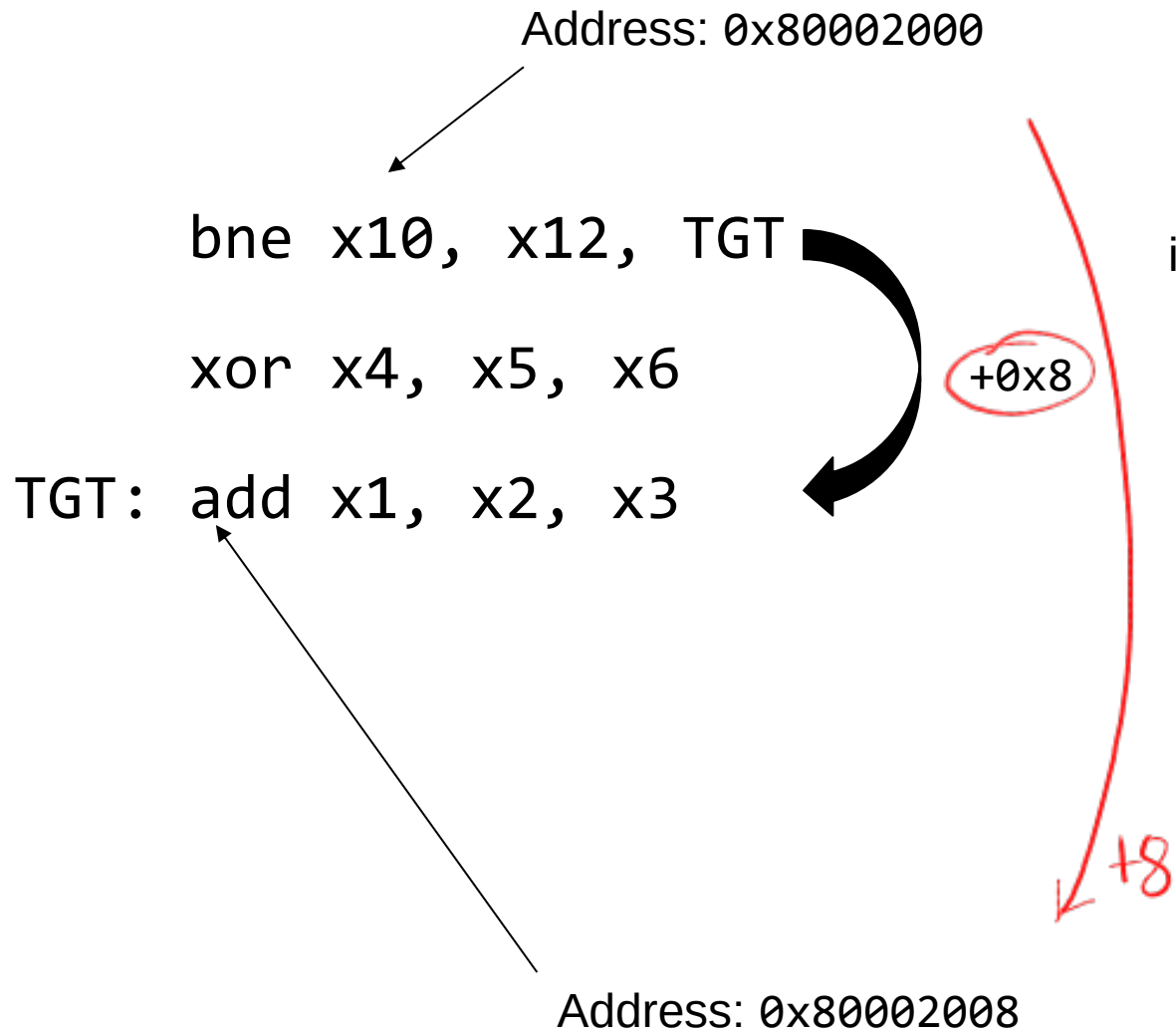
Branch Addressing

- Branch instructions specify:
 - ❖ Opcode, two registers, branch target
 - PC-relative addressing
 - ❖ Target address = PC + immediate
 - ❖ Forward or backward
 - Most branch targets are near the branch instruction
 - ❖ Immediate is a 13-bit signed value, and imm[0] is assumed to be zero.
- Handwritten notes in red:
- Immediate
12bit → 0000 ... 1001
actual offset
0000 ... 10010
13bit



Branch Encoding Example

beq : SB format



13bit

0000000001000₂ : 8₁₀

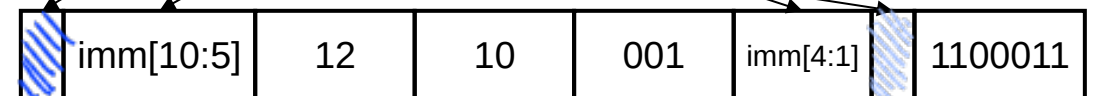
0000000001000₂ : imm [12:0]

000000000100₂ : imm [12:1]

12bit

0 0 000000 0100

0011 1000 0110 0011
32bit
12bit
12bit

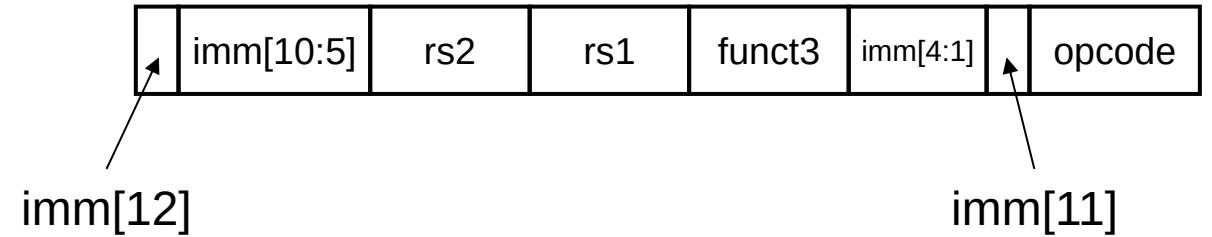
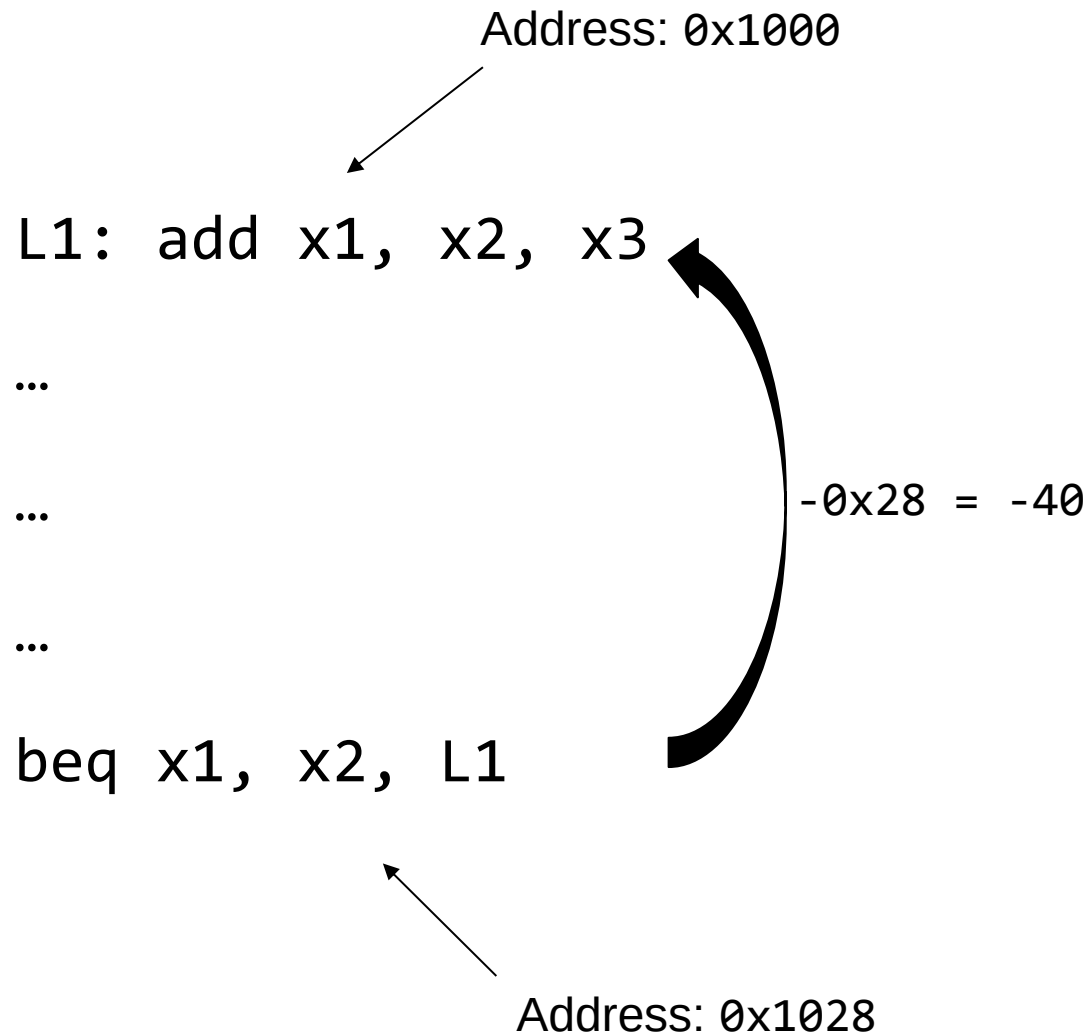


0 000000 01100 01010 001 0100 0 1100011₂

0000 0000 1100 0101 0001 0100 0110 0011₂

= 0x00C51463

Branch Encoding Example

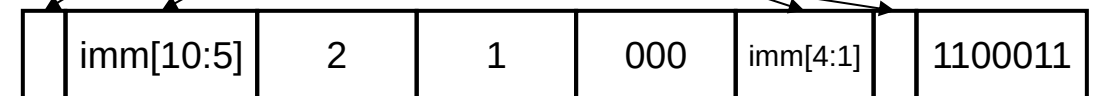


1111111011000₂ : -40₁₀

1111111011000₂ : imm [12:0] *실제값 1374*

111111101100₂ : imm [12:1] *비트값 1274*

1 1 111110 1100



1 111110 00010 00001 000 1100 1 1100011₂

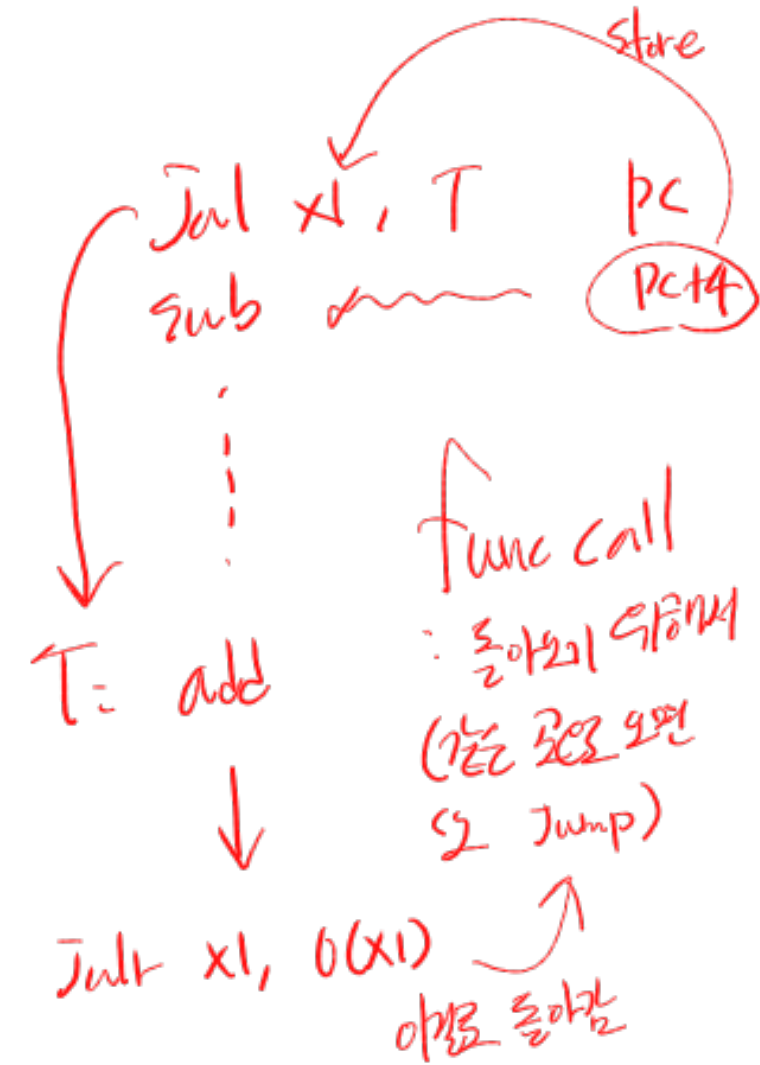
1111 1100 0010 0000 1000 1100 1110 0011₂

= 0xFC208CE3

RISC-V Jump Instructions

conditionX always unconditional

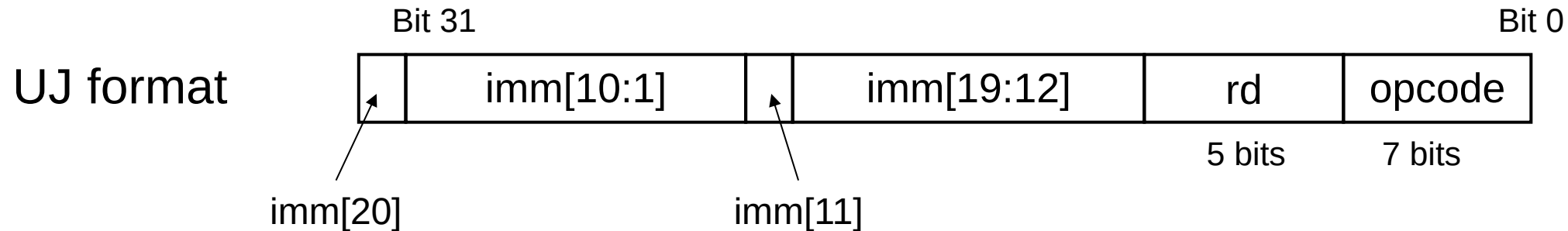
- **jal x1, L1** : jump-and-link
 - ❖ Jump to L1
 - ❖ Address of the following instruction (PC+4) → x1
- **jalr x1, offset(x2)** : jump register
 - ❖ Jump to offset + address in x2
 - ❖ Address of the following instruction (PC+4) → x1
- **j L1** : jump
 - ❖ Pseudo-instruction
 - ❖ jal x0, L1



Jump Addressing

- Jump and link (jal) target uses a 20-bit immediate for a larger range

21 bit Immediate 24 bits 20 bits
branch 13 bit Immediate 24 bits 12 bits



- For long jumps (e.g., to 32-bit absolute address),
 - ❖ lui: load address[31:12] to a temporary register
 - ❖ jalr: add address offset [11:0] to the temporary register and jump to target

RISC-V Encoding Summary

R format

funct7	rs2	rs1	funct3	rd	opcode
--------	-----	-----	--------	----	--------

I format

immediate	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------

S format

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
-----------	-----	-----	--------	----------	--------

SB format

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
---------	-----------	-----	-----	--------	----------	---------	--------

UJ format

imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
---------	-----------	---------	------------	----	--------

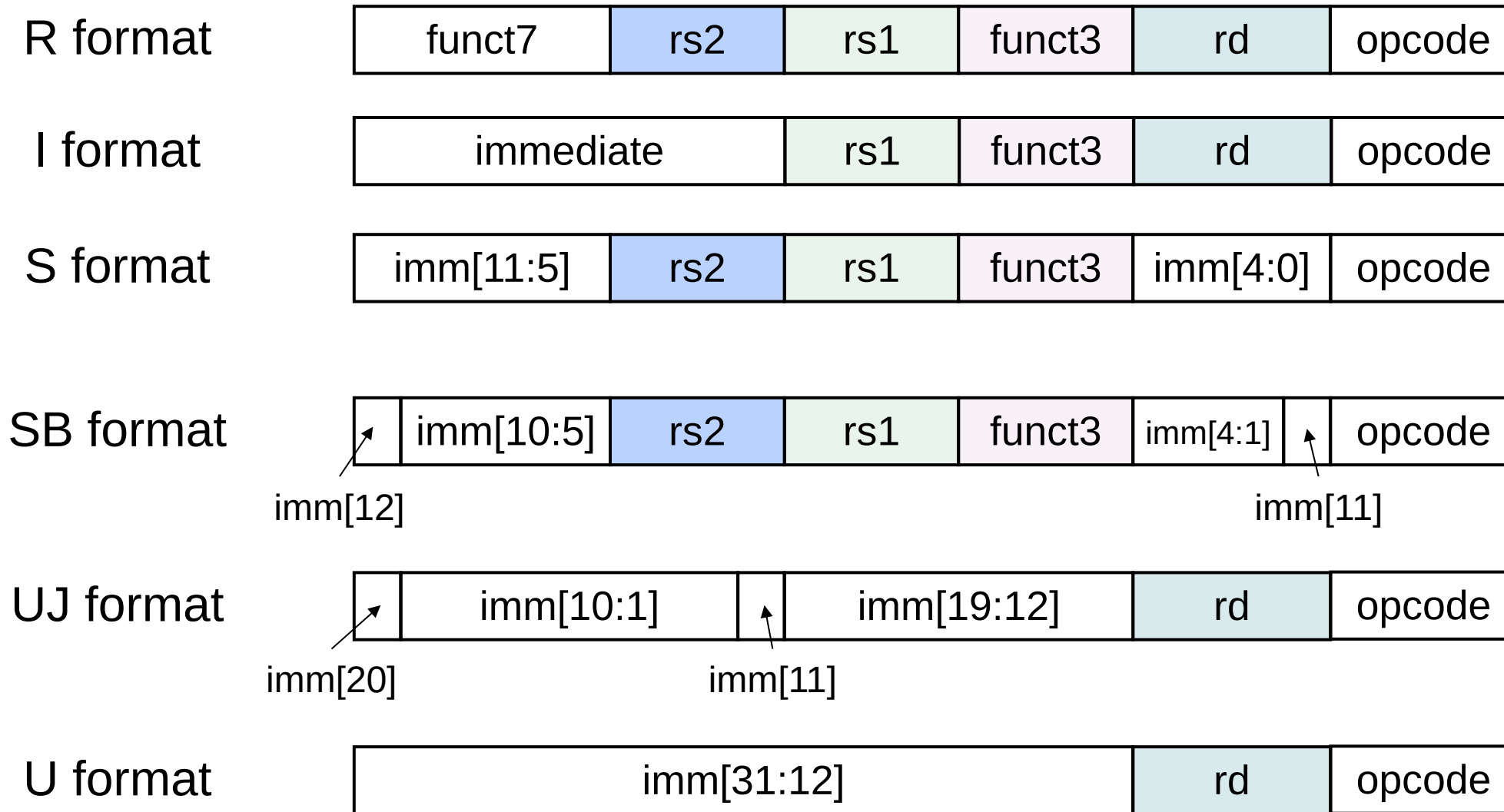
U format

imm[31:12]	rd	opcode
------------	----	--------

RISC-V Encoding Summary

R format	funct7	rs2	rs1	funct3	rd	opcode
I format	immediate		rs1	funct3	rd	opcode
S format	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
SB format	imm[12] imm[10:5] rs2 rs1 funct3 imm[4:1] imm[11] opcode					
UJ format	imm[20] imm[10:1] imm[11] imm[19:12] rd opcode					
U format	imm[31:12]				rd	opcode

RISC-V Encoding Summary



RISC-V Encoding Summary

 :Instruction's MSB is the MSB of the immediate value (for sign extension)

I format



12-bit imm. [11:0]

S format



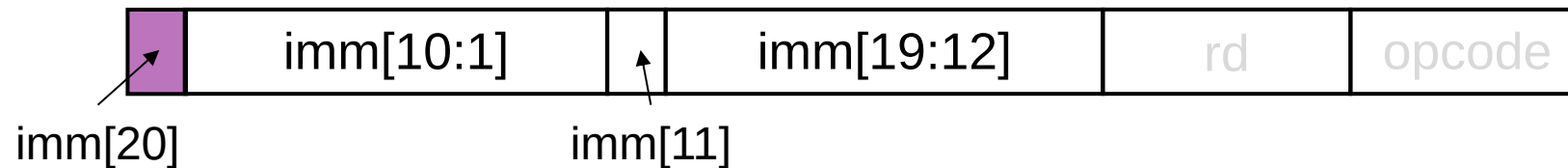
12-bit imm. [11:0]

SB format



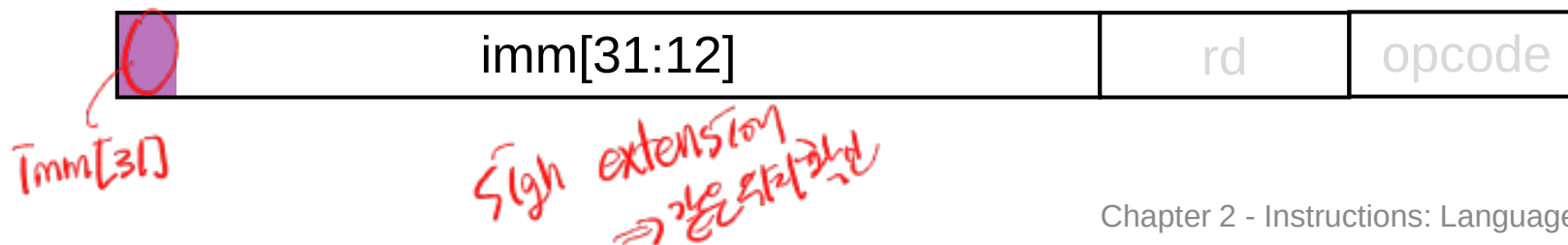
12-bit imm. [12:1]

UJ format



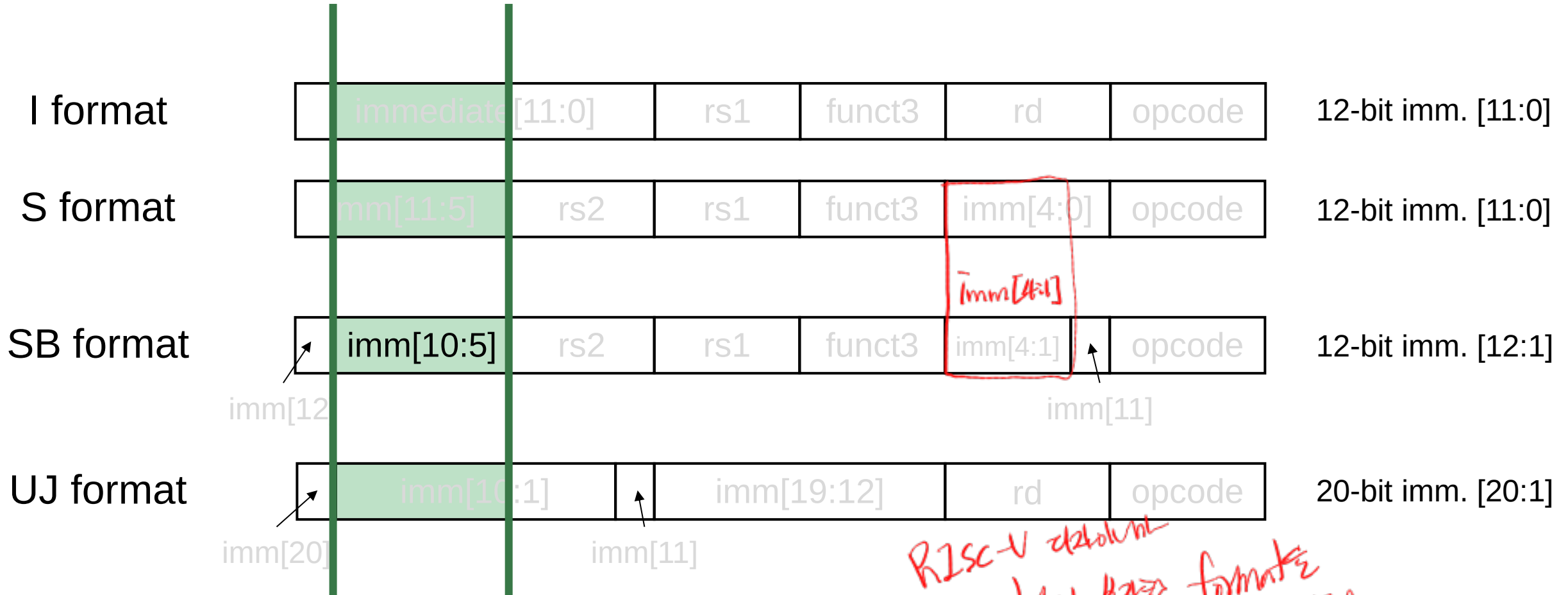
20-bit imm. [20:1]

U format



20-bit imm. [31:12]

RISC-V Encoding Summary



U format imm [10:5] X
7E location

RISC-V 24bit
이렇게 24bit format을
쓰고 7E로 30bit location

Aligned vs. Non-aligned Immediate Values

All instructions w/ imm[10:5] :



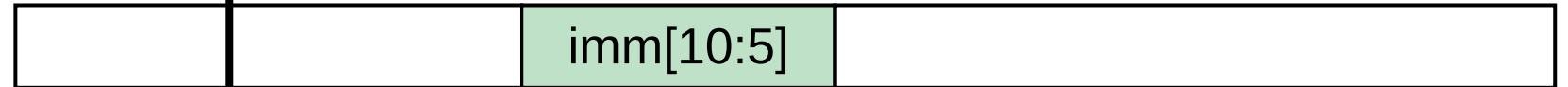
Immediate value:



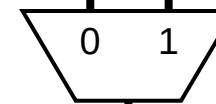
Instruction Type A :



Instruction Type B :



Instruction Type==B?



Immediate value:



다
구분
가능

3비트
MUX → 4비트